CS221 Autumn 2025: Artificial Intelligence: Principles and Techniques

Homework 3: Route Planning

SUNet ID: [your SUNet ID]
Name: [your first and last name]
Collaborators: [list all the people you worked with]

By turning in this assignment, I agree by the Stanford honor code and declare that all of this is my own work.

In route planning, the objective is to find the best way to get from point A to point B (think Google Maps). In this homework, we will build on top of the classic shortest path problem to allow for more powerful queries. For example, not only will you be able to explicitly ask for the shortest path from the Gates Building to the Green Library Coupa Cafe, but you can ask for the shortest path from Gates back to your dorm, stopping by the package center, gym, and the dining hall (in any order!) along the way.

Before you get started, please read the Assignments section on the course website thoroughly.

Problem 1: Grid City

Consider an infinite city consisting of locations (x, y) where x, y are integers. From each location (x, y), one can go east, west, north, or south. These movements are what we refer to as 'actions'. Specifically, an action is a change in the (x, y) coordinates: (+1, 0) represents moving east, (-1, 0) represents moving west, (0, +1) represents moving north, and (0, -1) represents moving south.

You start at $(0, 0)$ and want to go to (m, n), where $m, n \geq 0$. We can define the following search problem to capture this:

1. $s_{\text{start}} = (0, 0)$
2. $\text{Actions}(s) = \{(+1, 0), (-1, 0), (0, +1), (0, -1)\}$
3. $\text{Succ}(s, a) = s + a$
4. $\text{Cost}((x, y), a) = 1 + \max(x, 0)$ (i.e., it is more expensive as x increases)
5. $\text{IsEnd}(s) = \mathbf{1}[s = (m, n)]$

Part 1a: Calculating Minimum Cost

Question: What is the minimum cost of reaching location (m, n) (note that $m, n \geq 0$) starting from location $(0, 0)$ in the above city? Describe one possible path achieving the minimum cost. Is it unique (i.e., are there multiple paths that achieve the minimum cost)?

What we expect: Provide an expression for the minimum cost and explain how you got it. In addition, please provide 1 - 2 sentences describing one possible minimum cost path and state whether it is unique.

Your Solution: The minimum cost is $n + \frac{m(m+1)}{2}$. Note that the cost depends on the x-coordinate of the current state, so we should move upward when x is 0, where the minimum cost lies. Since the path is $(0, 0) \to (0, n) \to (m, n)$, it's unique.

Part 1b: UCS Behavior

Question: How will Uniform Cost Search (UCS) behave on this problem? Mark the following as true or false:

1. UCS will never terminate because the number of states is infinite.
2. UCS will return the minimum cost path and explore only locations between $(0, 0)$ and (m, n); that is, (x, y) such that $0 \leq x \leq m$ and $0 \leq y \leq n$
3. UCS will return the minimum cost path and explore only locations whose past costs are less than or equal to the minimum cost from $(0, 0)$ to (m, n).

What we expect: T/F for each subpart. No justification needed.

Your Solution:
1.F
2.F
3.T

Part 1c: UCS on an Arbitrary Graph

Question: Now consider running UCS on an arbitrary graph. Mark the following as true or false:

1. If you add a connection between two locations, the minimum cost cannot go up.
2. If you make the cost of an action from some state small enough (possibly negative), that action will show up in the minimum cost path.

3. If you increase the cost of each action by 1, the minimum cost path does not change (even though its cost does).

What we expect: T/F for each subpart. No justification needed.

Your Solution:
1.T
2.F
3.F

Problem 2: Finding Shortest Paths

We first start out with the problem of finding the shortest path from a start location (e.g., the Gates Building) to some end location. In Google Maps, you can only specify a specific end location (e.g., Coupa Cafe at Green Library).
In this problem, we want to give the user the flexibility of specifying multiple possible end locations by specifying a set of "tags" (e.g., so you can say that you want to go to any place with food versus a specific location like Tressider).

Part 2b: Get Stanford Shortest Path Problem

Question: Run python map_util.py > readable_stanford_map.txt to write a (long-ish) file of the possible locations on the Stanford map along with their tags. Each tag is a [key]=[value]. Here are some examples of keys:

- landmark: Hand-defined landmarks (from data/stanford-landmarks.json)
- amenity: Various amenity types (e.g., "parking_entrance", "food")
- parking: Assorted parking options (e.g., "underground")

Choose a starting location and end tag (perhaps that's relevant to your daily life) and implement get_stanford_shortest_path_problem() to create a search problem. Then, run python grader.py 1b-custom to generate path.json. Once generated, run python visualization.py to visualize it (opens in browser). Try different start locations and end tags. Pick two settings corresponding to the following:

- A start location and end tag that produced new insight into traveling around campus. Describe whether the system was useful.
- A start location and end tag where the minimum cost path found isn't desirable. Is this due to incorrect modeling assumptions? Explain.

You should feel free to add new landmarks. If you are not familiar with the Stanford campus, please feel free to use the Campus Map website to learn more about buildings, amenities, and paths around campus.

What we expect: A screenshot of the visualization of your two solutions as well as i) one or more sentences describing something interesting you've learned about traveling around campus (or elsewhere) and ii) something incorrect about either the map or modeling assumptions (such as a landmark being out of place, etc.).

Your Solution: NOT FINISHED since I'm not a Stanford student. But the process of visualization is quite funny. Below is an example:

Part 2c: Negative Externalities

Question: Your system now allows anyone to find the shortest path between any pair of locations on campus. By shortening their travel distance, it promises to optimize travel efficiency. But, there might be issues when a large fraction of the population start using your system to plan their routes.

In particular, what *negative externalities* might result from this system being widely deployed? Please refer to these brief articles for inspiration: "Why Traffic Apps Make Congestion Worse," "The Perfect Selfishness of Mapping Apps", as well as the module on Externalities (video and pdf).

Discuss the impact of these externalities on users of your system and other people. Remember that these sorts of problems arise from the mismatch between the real world and one's model of it.

What are potential ways you could reduce this mismatch?

What we expect: Provide 3-5 sentences describing (a) two externalities (one for users of the route-planning system and one for other people/non-users of the system), as well as (b) at least one potential solution to reduce the impact of these problems.

Your Solution:

1) For system users: When a large number of users follow the same "shortest path" recommended by the system, the origin path may become congested;

2) Their origin preferred path may become worse(more congested or more dirty), which creates a negative consumption since non-users bear the costs while not benifiting from the route-planning tool.

My suggestion: The route-planning tool should provide the funtion of visualizing the real-

time congestion, which will avoid those users followingthe same ”shortest path”.

Problem 3: Finding Shortest Paths with Unordered Waypoints

Let's introduce an even more powerful feature: unordered waypoints! In Google Maps, you can specify an ordered sequence of waypoints that a path must go through – for example, going from point A to point X to point Y to point B, where [X, Y] are "waypoints" (such as a gas station or a friend's house).
However, we want to consider the case where the waypoints are unordered: X, Y, so that both $A \to X \to Y \to B$ and $A \to Y \to X \to B$ are allowed. Moreover, X, Y, and B are each specified by a tag like in Problem 1 (e.g., amenity=food).
This is a neat feature if you think about your day-to-day life; you might be on your way home after a long day, but need to stop by the package center, Tressider to grab a bite of food, and the bookstore to buy some notebooks. Having the ability to get a short, quick path that hits all these stops might be really convenient (rather than searching over the various waypoint orderings yourself).

Part 3b: Maximum Number of States

Question: If there are n locations and k waypoint tags, what is the maximum possible number of states given a suitable state defintion for part a that UCS could visit?

What we expect: A mathematical expression that depends on n and k, with a brief explanation justifying it.

Your Solution:

Part 3c: Get Stanford Waypoints Shortest Path Problem

Question: Choose a starting location, set of waypoint tags, and an end tag (perhaps that captures an interesting route planning problem relevant to you), and implement get_stanford_waypoints_shortest_path_problem() to create a search problem. Then, similar to Problem 2b, run python grader.py 3c-custom to generate path.json. Once generated, run python visualization.py to visualize it (opens in browser).

What we expect: A screenshot of the visualized path with a list of the waypoint tags you selected. In addition, provide one or more sentences describing unexpected or interesting features of the selected path. Does it match your expectations? What does this path fail to capture that might be important?

Your Solution:

Part 3d: Ride Sharing

Question: Ride sharing companies use route-finding systems similar to the one you built in this problem to route their drivers. However, these systems have been criticized for using exploitative behavioral nudges. For example, a New York Times article reported how one app uses "forward dispatch," a feature that dispatches a new ride to a driver before the current one ends, to constantly keep drivers busy. This makes it more difficult for drivers to take breaks.

We want you to discuss how these companies could implement better labor practices, finding a way to use the unordering waypoints feature to balance company goals with the protection of driver interests. Specifically, we want you to think about ways your new unordered waypoints feature could help drivers in their working experiencing.

What waypoints could you include from one drop-off location to the next pick-up location to improve the driver experience? What information about drivers would you need to determine appropriate waypoints?

Note, however, that the unordered waypoints feature is an example of a dual use technology. For more, refer to the module on dual use technologies to help answer this question video and pdf).

Thus, we also want you to answer: What are some ways that a company could use this waypoints feature to increase profits at the expense of driver health? What are any downsides in collecting the appropriate information you need to recommend the waypoints?

What we expect: Provide 3-5 sentences describing (a) the waypoints you would include to improve the driver experience, (b) at least one dimension of drivers' identity that could inform selection of waypoints, and (c) a potential negative use of the waypoints feature that a company could use to increase profits while harming the driver experience or driver health.

Your Solution:

Problem 5: AI Tutor

Part 5a: Dynamic Programming

Question: Practice Dynamic Programming with an AI tutor! Use an AI chatbot (e.g., ChatGPT, Claude, Gemini, or the Stanford AI Playground).

What we expect: Provide a link to the chat session transcript with the AI tutor. The session should be 15–20 minutes and interactive.

Your Solution: Link:

Part 5b: Beam Search

Question: Practice Beam Search with an AI tutor! Use an AI chatbot (e.g., ChatGPT, Claude, Gemini, or the Stanford AI Playground).

What we expect: Provide a link to the chat session transcript with the AI tutor. The session should be 15–20 minutes and interactive.

Your Solution: Link:

Submission

Submission is done on Gradescope.

Written: When submitting the written parts, make sure to select **all** the pages that contain part of your answer for that problem, or else you will not get credit. To double check after submission, you can click on each problem link on the right side and it should show the pages that are selected for that problem.

Programming: After you submit, the autograder will take a few minutes to run. Check back after it runs to make sure that your submission succeeded. If your autograder crashes, you will receive a 0 on the programming part of the assignment. Note: the only file to be submitted to Gradescope is `submission.py`.

More details can be found in the Submission section on the course website.